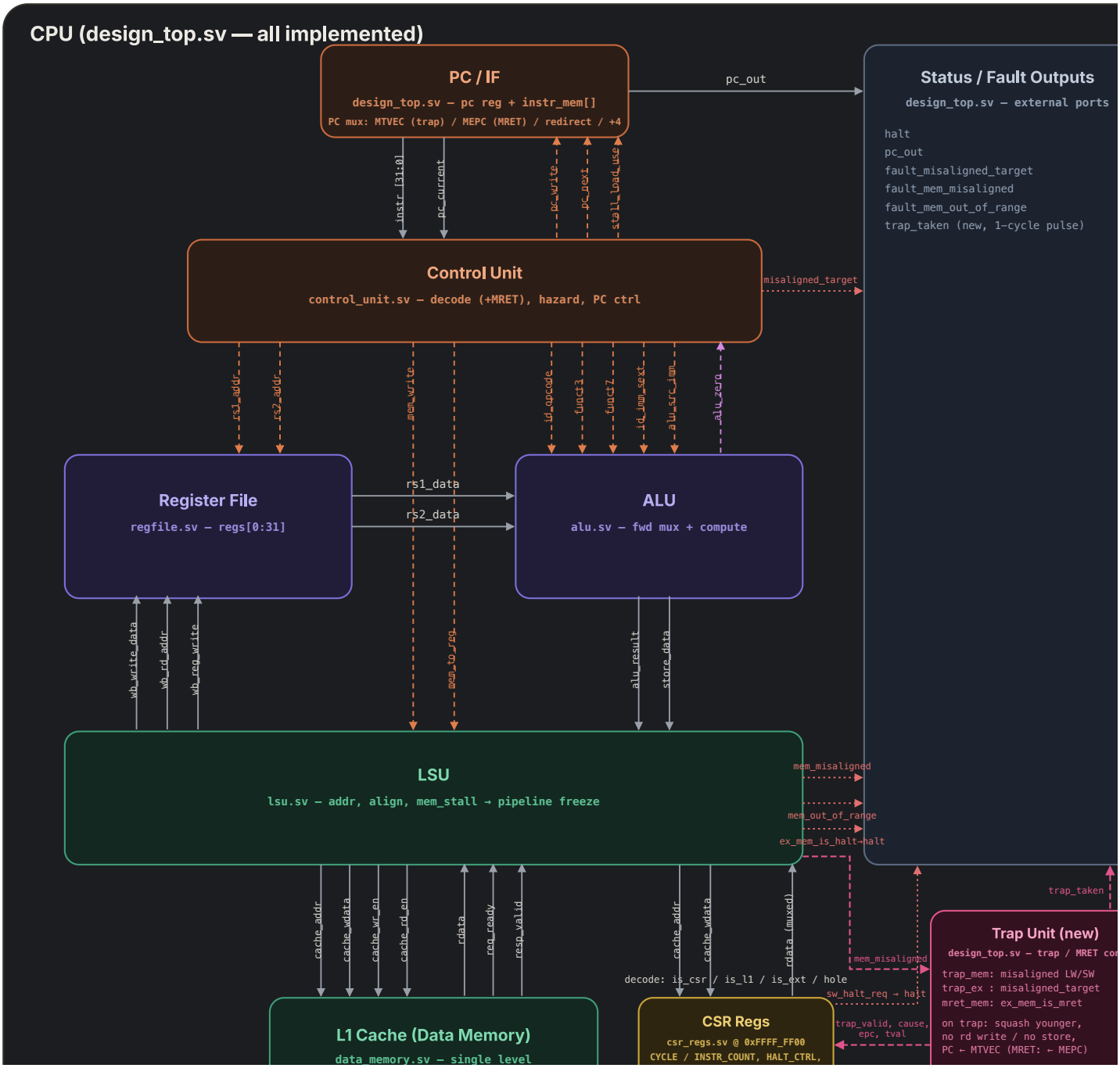
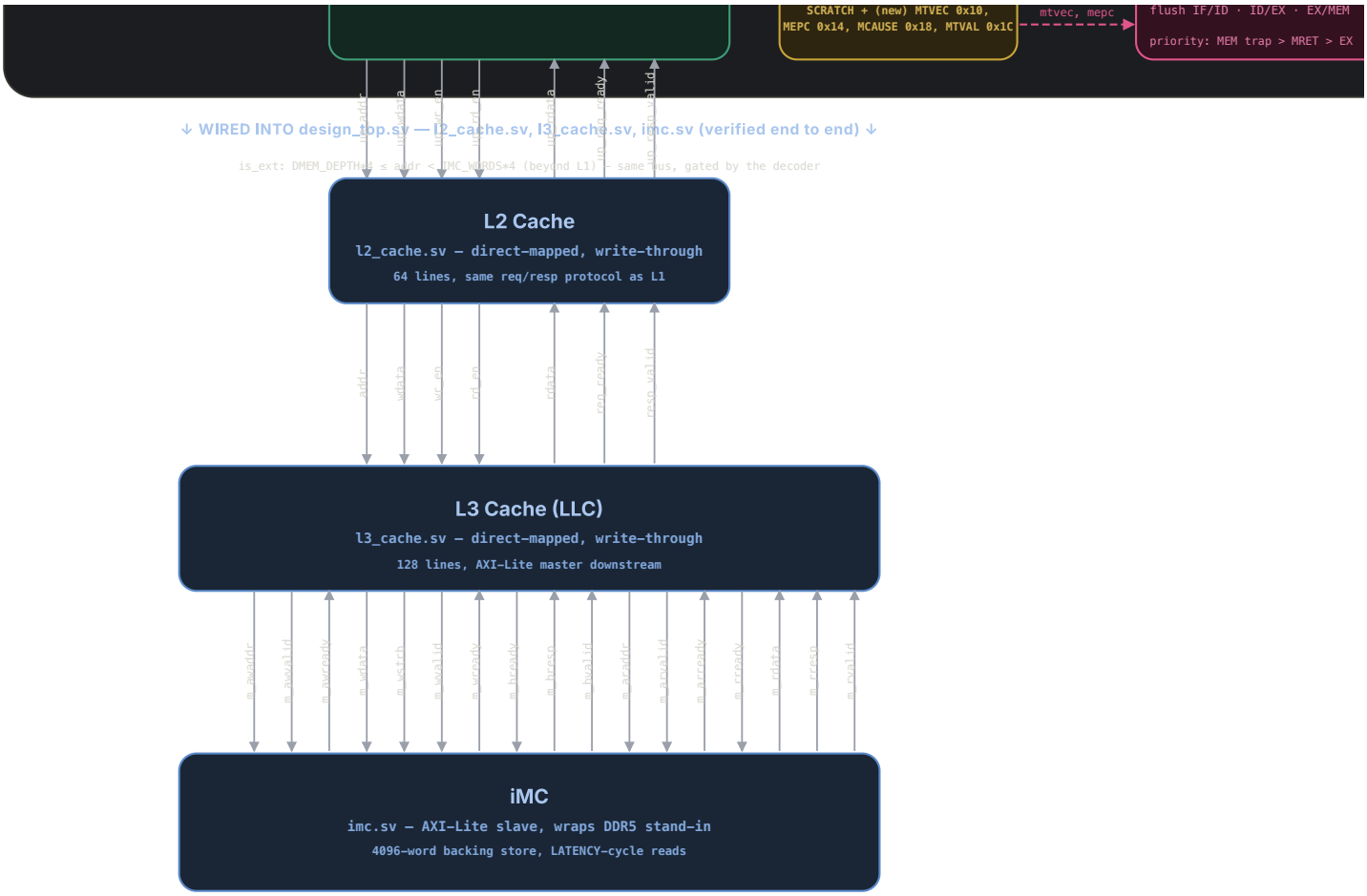
CPU DUT — Full Block Diagram

Every solid-outline block and arrow is real RTL: the CPU core off `control_unit.sv`, `regfile.sv`, `alu.sv`, `lsu.sv`, `data_memory.sv`, `design_top.sv`, and now **L2, L3, and iMC are real modules too** (`l2_cache.sv`, `l3_cache.sv`, `imc.sv`) — direct-mapped, write-through caches, with a genuine AXI-Lite link from L3 into iMC. All of it is now wired into `design_top.sv`: addresses beyond L1 flow through L2, L3 and AXI-Lite into iMC, and `mem_stall` freezes the pipeline while they answer. **New in this version:** misaligned loads/stores and misaligned jump/branch targets now **trap** RISC-V style instead of only raising a flag — a pink **Trap Unit** squashes the faulting instruction and everything younger, `csr_regs.sv` records `MEPC / MCAUSE / MTVAL` (and holds MTVEC), the PC jumps to the handler, and a new MRET instruction returns. See the trap-flow section below the diagram.





Control (dashed) Status feedback (dashed) Fault / exception (dashed)

Data bus (solid) CSR / address-mapped regs Trap path (new)

PC/IF, the **Status/Fault Outputs** block, and the `alu_zero` feedback are the pieces added on top of the earlier version — all real, off `design_top.sv`'s actual ports and pipeline registers.

New: CSR Regs (`csr_regs.sv`) sits beside L1 as a second backing store for the exact same `cache_addr`/`cache_wdata`/`cache_wr_en`/`cache_rd_en` bus LSU already drives — LSU itself never changes. A small decoder in `design_top.sv` (`is_csr = cache_addr ≥ CSR_BASE = 0xFFFF_FF00`) gates which store's `wr_en`/`rd_en` actually fire and muxes `rdata`/`req_ready`/`resp_valid`/`misaligned`/`out_of_range` back from whichever one was hit. The block holds four registers — `CYCLE_COUNT` and `INSTR_COUNT` (read-only counters), `SCRATCH` (plain read/write), and `HALT_CTRL`, whose `sw_halt_req` output ORs into the same sticky `halt` flop `ex_mem_is_halt` already sets — so a program can halt itself with a plain `SW`, no `ECALL` required. Reached with ordinary `LW`/`SW` once an address is in a register (here, no `LUI` even needed — `0xFFFF_FF00` is just `-256` sign-extended); never confused with `regfile.sv`'s GPRs, which stay reachable only through `rs1`/`rs2`/`rd` and were never put on this bus.

Memory map, in address order: `[0, DMEM_DEPTH*4)` is L1 (`data_memory.sv`); `[DMEM_DEPTH*4, IMC_WORDS*4)`

goes L2 → L3 → AXI-Lite → iMC; `[CSR_BASE, …)` is `csr_regs.sv`; anything else is a hole and raises `fault_mem_out_of_range`. Misaligned accesses beyond L1 are stopped before they reach L2, raise `fault_mem_misaligned` **and now take a trap** (cause 4/6). L1 here is directly addressed on-chip RAM with no tags, so "beyond L1" is decided by address range, not by a tag miss.

Pipeline stall: `lsu.sv` already produced `mem_stall`, but nothing in `design_top.sv` consumed it, which would have corrupted the pipeline the moment a level took more than one cycle. It now freezes the PC and every pipeline register (MEM/WB included, so forwarding sources stay valid) until the hierarchy answers. The CSR instruction counter ignores the repeated cycles.

Verified: a directed test stores and loads through the full chain (data lands in `iMC`), runs a load-use hazard on a stalled load, shows a second read hitting L2 with zero stall cycles, confirms L1 accesses never stall, and confirms an access into a hole in the map faults without hanging. The earlier L1/CSR tests still pass. Note: the original `l2_cache.sv`, `l3_cache.sv` and `imc.sv` were lost when the working container reset, so they were rebuilt to the same spec shown here (64 and 128 one-word lines, direct-mapped, write-through, AW→W→B and AR→R sequenced one channel at a time).

Halt — now stops the CPU

`halt` is set when `ECALL` (or a `SW` of 1 to `HALT_CTRL`) is in MEM. In that same cycle every younger instruction is squashed and the PC freezes; older instructions and the halting instruction itself still complete, and a store still in flight to L2/L3/iMC finishes first. Nothing after it can write a register, write memory or move the PC. `instr_count` stops counting. Only reset clears it. Verified by `tb_halt.sv` (ECALL case with a poison taken branch behind it, software-halt case behind a stalled external store, and a reset-and-rerun); the same test fails on the old flag-only version.

Misalignment trap — how it works

RISC-V style **precise trap**: the faulting instruction never completes and nothing younger survives. Only misalignment traps; out-of-range (hole in the map) still just raises `fault_mem_out_of_range`. No nested-trap stack: a fault inside the handler overwrites `MEPC`.

1 Detect. A LW/SW reaches MEM with `addr[1:0] ≠ 00` (`mem_misaligned`), or a taken branch/JAL in EX has a target with `[1:0] ≠ 00`.

2 Cancel. No register write (`mem_wb_reg_write` gated), no store (memory blocks misaligned writes), the instruction does not count as retired. EX/MEM, ID/EX and IF/ID are squashed.

3 Record. `csr_regs` latches MEPC = faulting PC, MCAUSE = 0 / 4 / 6, MTVAL = bad address or jump target. `trap_taken` pulses.

4 Vector. PC ← MTVEC (set earlier by SW to `0xFFFF_FF10`; reset value 0).

5 Handle. Handler code reads MCAUSE/MTVAL/MEPC with LW, adds 4 (or 8) to MEPC and stores it back with SW.

6 Return. MRET (`0x30200073`) resolves in MEM, so a SW to MEPC the instruction before has already landed. PC ← MEPC, younger stages flushed.

MCAUSE	Meaning	Detected in	MEPC	MTVAL
0	jump/branch target misaligned	EX	PC of the branch/JAL	the bad target
4	load address misaligned	MEM	PC of the LW	the bad address
6	store address misaligned	MEM	PC of the SW	the bad address

Example: `x5 = 0x100`, `LW x6, 6(x5)` → address `0x106`, low bits `10` → trap with `MCAUSE=4`, `MTVAL=0x106`, `MEPC=` the LW's PC; `x6` keeps its old value. The handler adds 4 to MEPC and `MRET`s to the next instruction. Cost: the faulting instruction plus 3 younger slots are lost on entry, and 3 bubbles on `MRET`.

Verified: `tb_trap.sv` fires six traps (misaligned store in L1, load in L1, load beyond L1, `JAL` target, taken `BEQ` target, load in the CSR window). It checks MCAUSE/MTVAL/MEPC for each, that the poison instruction after every fault is squashed, that no store or register write happens, that the instruction before the fault still commits, and the back-to-back `SW MEPC; MRET` case. Deliberately breaking the squash, the rd-write gate or the flush each makes it fail. The earlier three testbenches still pass.